

Experiment 7: Selection Sort on Random, Reverse and Sorted Arrays

Aim: To implement Selection Sort and compare its performance on random, reverse-sorted, and already-sorted arrays.

Algorithm

1. Create three arrays: random, reverse-sorted, and sorted.
2. Find the smallest element in the unsorted part of the array.
3. Swap it with the first element of the unsorted part.
4. Repeat until the complete array is sorted.
5. Display the sorted arrays.
6. Count the number of comparisons for each type of array.

Python Program

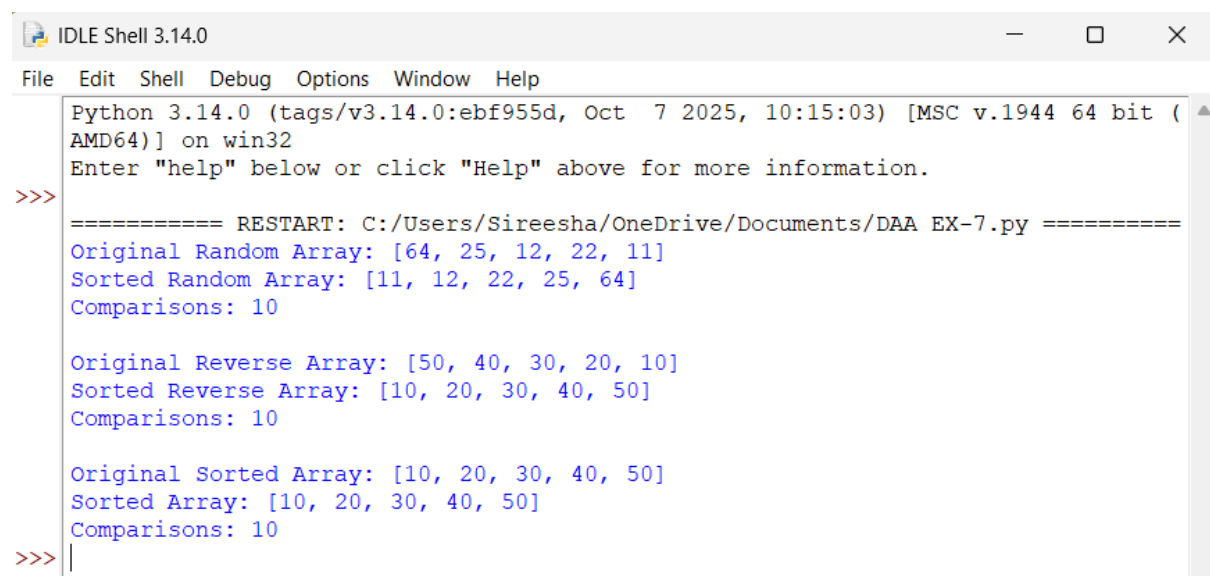
```
def selection_sort(arr):  
    comparisons = 0  
    n = len(arr)  
    for i in range(n - 1):  
        min_index = i  
        for j in range(i + 1, n):  
            comparisons += 1  
            if arr[j] < arr[min_index]:  
                min_index = j  
        arr[i], arr[min_index] = arr[min_index], arr[i]  
    return comparisons  
  
random_array = [64, 25, 12, 22, 11]  
reverse_array = [50, 40, 30, 20, 10]  
sorted_array = [10, 20, 30, 40, 50]  
print("Original Random Array:", random_array)  
c1 = selection_sort(random_array)  
print("Sorted Random Array:", random_array)  
print("Comparisons:", c1)
```

```

print()
print("Original Reverse Array:", reverse_array)
c2 = selection_sort(reverse_array)
print("Sorted Reverse Array:", reverse_array)
print("Comparisons:", c2)
print()
print("Original Sorted Array:", sorted_array)
c3 = selection_sort(sorted_array)
print("Sorted Array:", sorted_array)
print("Comparisons:", c3)

```

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/DAA EX-7.py =====
Original Random Array: [64, 25, 12, 22, 11]
Sorted Random Array: [11, 12, 22, 25, 64]
Comparisons: 10

Original Reverse Array: [50, 40, 30, 20, 10]
Sorted Reverse Array: [10, 20, 30, 40, 50]
Comparisons: 10

Original Sorted Array: [10, 20, 30, 40, 50]
Sorted Array: [10, 20, 30, 40, 50]
Comparisons: 10
>>>

```

Result

Thus, the Selection Sort algorithm was successfully implemented and its performance on random, reverse-sorted, and sorted arrays was analyzed.